

JavaScript: a tela começa a responder

Lista de exercícios - Dia 1

Você vai construir cada página desde o zero. O HTML e o CSS ficam por sua conta; o objetivo avaliado hoje é o comportamento em JavaScript.

Regra principal

Não copie o projeto pronto antes de tentar. O repositório da turma é uma referência para quando você travar, não um arquivo inicial obrigatório.

O que usamos hoje

- ☐ `document.getElementById(...)` para achar um elemento.
- ☐ `addEventListener('click', ...)` para escutar um clique.
- ☐ `textContent` para mudar um texto na tela.
- ☐ `const` para guardar uma referência a um elemento.
- ☐ `let` somente quando um valor precisar mudar, como um contador.

ACHAR → ESCUTAR → FAZER

1. Ache no JavaScript o elemento criado no HTML.
2. Escute a ação do usuário.
3. Faça alguma mudança visível na página.

Antes de começar

- ☐ Crie uma pasta separada para cada exercício.
- ☐ Em cada pasta, crie `index.html`, `estilo.css` e `script.js`.
- ☐ Conecte o CSS e o JavaScript ao HTML.
- ☐ Abra a página no navegador e mantenha o Console disponível.
- ☐ Dê nomes claros aos ids. Evite ids como `botao1` ou `texto2`.

Exercício 1 - Primeiro clique

Objetivo: fazer um botão alterar uma mensagem da página.

Construa uma página com:

- ☐ Um título de sua escolha.
- ☐ Um parágrafo que comece com Nenhuma ação realizada.
- ☐ Um botão com o texto Clique aqui.

Comportamento obrigatório

- ☐ Quando o botão for clicado, o parágrafo deve mudar para O botão foi clicado!
- ☐ A mudança precisa acontecer sem recarregar a página.
- ☐ O JavaScript deve estar em um arquivo separado.

Contrato entre HTML e JavaScript

```
<!-- Você escolhe os nomes, mas os ids devem existir -->
<p id="mensagem">Nenhuma ação realizada.</p>
<button id="botaoAcao">Clique aqui</button>
```

Estrutura mental do JavaScript

```
const botao = document.getElementById('...');
const mensagem = document.getElementById('...');

botao.addEventListener('click', function () {
  mensagem.textContent = '...';
});
```

Pista 1	Confira se os ids usados no JavaScript são exatamente iguais aos ids do HTML.
Pista 2	Abra o Console. Se aparecer “Cannot read properties of null”, o JavaScript não encontrou algum elemento.
Pista 3	Confira se a tag <script> aponta para o arquivo correto e se foi colocada no fim do <body> ou usa defer.

Terminou?

Mude a frase final e o texto do botão sem alterar a lógica.

Exercício 2 - Três escolhas, uma resposta

Objetivo: repetir a mesma receita com três botões diferentes.

Tema sugerido: escolha de período

- ☐ Crie os botões Manhã, Tarde e Noite.
- ☐ Crie um texto de resumo começando com Período escolhido: nenhum.

Comportamento obrigatório

- ☐ Manhã → Período escolhido: manhã.
- ☐ Tarde → Período escolhido: tarde.
- ☐ Noite → Período escolhido: noite.

Restrições

- ☐ Use apenas um parágrafo de saída.
- ☐ Cada botão precisa ter seu próprio evento de clique.
- ☐ Não use alert(). A resposta deve aparecer dentro da página.

Planejamento antes de programar

Elemento	id sugerido	O que acontece
Botão Manhã	periodoManha	Atualiza o resumo
Botão Tarde	periodoTarde	Atualiza o resumo
Botão Noite	periodoNoite	Atualiza o resumo
Resumo	periodoEscolhido	Recebe o novo texto

Pista 1	Ache os quatro elementos antes de criar os eventos.
Pista 2	Faça primeiro apenas o botão Manhã. Teste. Depois repita para os outros.
Pista 3	Se todos os botões mostram a mesma frase, confira o texto dentro de cada função.

Desafio opcional

Troque o tema para algo seu: dificuldade de jogo, música, personagem, cor ou esporte.

Exercício 3 - Contador de cliques

Objetivo: guardar um valor que muda e mostrá-lo na tela.

Construa uma página com:

- ☐ Um número começando em 0.
- ☐ Um botão Adicionar 1.
- ☐ Um botão Zerar.

Comportamento obrigatório

- ☐ Cada clique em Adicionar 1 aumenta o número em uma unidade.
- ☐ O botão Zerar volta o número para 0.
- ☐ O valor visível sempre corresponde ao valor guardado no JavaScript.

Nova ideia: let

Use `const` para elementos que continuam apontando para o mesmo lugar. Use `let` para o número que será alterado durante a execução.

Pseudocódigo

guardar contador começando em 0

quando clicar em "Adicionar 1":
 aumentar contador
 mostrar contador na tela

quando clicar em "Zerar":
 voltar contador para 0
 mostrar contador na tela

Trecho permitido como referência

```
let contador = 0;
```

```
contador = contador + 1;  
elementoNumero.textContent = contador;
```

Pista 1	O número guardado e o número exibido são coisas diferentes. Depois de mudar a variável, atualize também a tela.
Pista 2	No evento de zerar, atribua 0 à variável antes de atualizar o texto.
Pista 3	Teste: adicionar, adicionar, adicionar, zerar, adicionar.

Desafio opcional

Adicione um botão Remover 1. Decida se o contador poderá ficar negativo.

Exercício 4 - Painel de status

Objetivo: combinar vários elementos e deixar claro qual estado está ativo.

Construa uma página chamada Status do sistema com:

- ☐ Três botões: Disponível, Ocupado e Offline.
- ☐ Um texto principal: Status atual: não definido.
- ☐ Um texto secundário: Mensagem: escolha um status.

Comportamento obrigatório

- ☐ Disponível: Disponível e Pode receber novas tarefas.
- ☐ Ocupado: Ocupado e Finalize a tarefa atual primeiro.
- ☐ Offline: Offline e Sem conexão no momento.

A etapa FAZER agora possui duas instruções

```
botao.addEventListener('click', function () {  
    textoStatus.textContent = '...';  
    textoMensagem.textContent = '...';  
});
```

Pista 1	No começo do arquivo, ache os três botões e os dois textos.
Pista 2	Implemente e teste um status por vez.
Pista 3	Se apenas uma frase muda, confira se as duas atribuições estão dentro da mesma função.

Desafio opcional

Use CSS para dar uma aparência diferente a cada status. O JavaScript obrigatório continua sendo a troca dos textos.

Exercício 5 - Monte seu Lanche

Objetivo: construir do zero a primeira etapa do projeto da semana. O repositório mostra uma possível solução completa, mas você deve criar a sua interface.

Parte visual - liberdade com responsabilidade

- ☐ Crie uma página com o título Monte seu Lanche.
- ☐ Crie três botões: Simples, Duplo e Triplo.
- ☐ Crie um resumo começando com Tamanho escolhido: nenhum.
- ☐ Organize e estilize a página usando o HTML e o CSS que você já conhece.

Comportamento JavaScript obrigatório

- ☐ Simples → Tamanho escolhido: Simples.
- ☐ Duplo → Tamanho escolhido: Duplo.
- ☐ Triplo → Tamanho escolhido: Triplo.
- ☐ A página não deve recarregar durante as escolhas.

Critérios de conclusão

- ☐ Todos os ids usados no JavaScript existem no HTML.
- ☐ O arquivo script.js está conectado corretamente.
- ☐ Não há erros vermelhos no Console.
- ☐ Os três botões funcionam em qualquer ordem.
- ☐ O código usa nomes compreensíveis.
- ☐ O resultado deixa claro o tamanho atual.

O que NÃO entra hoje

Preço, adicionais, campos de formulário, localStorage, instalação como aplicativo e funcionamento offline. Esses movimentos entram nas próximas aulas.

Pistas graduais

Pista 1	Pense em quatro elementos JavaScript: três botões e um texto de resumo.
Pista 2	Faça primeiro o botão Simples funcionar completamente.
Pista 3	Depois repita a mesma estrutura para Duplo e Triplo.
Pista 4	Somente após tentar, compare sua solução com a pasta dia-01 do repositório.

Quando não funcionar: roteiro de depuração

Não apague tudo. Descubra em qual etapa a receita quebrou.

Sintoma	Verificação
Nada acontece ao clicar	O evento foi criado? O botão foi encontrado? O script está conectado?
Erro com null no Console	O id do JavaScript não existe no HTML ou o script executou cedo demais.
A frase aparece errada	Confira a string colocada em <code>textContent</code> .
Só um botão funciona	Confira se cada botão tem seu próprio <code>addEventListener</code> .
A página recarrega	Se o botão estiver em um form, use <code>type="button"</code> quando necessário.
Alterei o JS e nada mudou	Salve o arquivo, recarregue a página e confira se está editando a pasta correta.

Autoavaliação do Dia 1

- ☐ Consigo explicar para que serve um id.
- ☐ Consigo achar um elemento com `getElementById`.
- ☐ Consigo criar um evento de clique.
- ☐ Consigo alterar um texto com `textContent`.
- ☐ Consigo usar `const` para guardar elementos.
- ☐ Consigo usar `let` quando um valor precisa mudar.
- ☐ Consigo ler um erro básico no Console.
- ☐ Consigo criar uma pequena interação sem copiar uma solução inteira.

Entrega sugerida

Entregue a pasta do Exercício 5 com `index.html`, `estilo.css` e `script.js`. Os exercícios 1 a 4 funcionam como treino progressivo.

Referência técnica do curso: repositório UC08, pasta dia-01. Use a implementação pronta apenas como ajuda após a tentativa.